DUNGEONS & DRAGONS

MULTIPLAYER BACKEND

Frontend Integration Guide

Backend Version	API Gateway URL	WebSocket URL
v1.0.0	`http://localhost:8080`	`ws://localhost:8080/ws`

> *The Golden Rule: You only ever communicate with one server — The Bouncer (port 8080). All other microservices (Vault, Referee, Lobby, Scribe) are internal and invisible to the frontend.*

Overview & Architecture

The backend is built as a distributed microservice system, but the frontend never needs to know or care about any internal service except The Bouncer. Every HTTP request and every WebSocket message flows through the Bouncer, which routes internally to the appropriate engine.

Service	Role & Notes
`Bouncer`	Port 8080 — API Gateway, Auth, WebSocket Hub. Your only point of contact.
`Lobby Manager`	Port 8081 — Room creation and matchmaking. Called by Bouncer internally.
`Referee`	Port 8082 — Game engine and turn validation. Called by Bouncer on each action.
`Vault`	Port 8083 — PostgreSQL database authority. Never called directly from frontend.
`Scribe`	Port 8084 — Post-game XP and leaderboard. Runs async after game ends.

The communication lifecycle for a typical game action looks like this:

```
Client (your app)
    |
```

```
    ├─ REST POST /api/game/action  ─────────────────┐
    │  OR STOMP /app/action (preferred)              │
    │                                                ▼
    │                                          [ Bouncer ]
    │                                                │
    │                                ┌───────────────┤
    │                                │               │
    │                                ▼               ▼
    │                          [ Referee ]      [ Vault ]
    │                          computes         persists
    │                          turn result      turn log
    │                                │
    │                                ▼
    │                      Redis Pub/Sub broadcast
    │                                │
    └──── /topic/match/{matchId} ◄──┘
         All 4 players receive TURN_RESULT instantly
```

□ Section 1: Authentication

The game uses stateless JWT (JSON Web Token) authentication. Every request must prove the player's identity. The process is a two-step handshake: get a Google ID token, then trade it for a game-specific JWT.

Step 1 — Get the Google ID Token

Install the Google OAuth library for React and render a login button. When the user completes Google's popup flow, Google hands your app an idToken string.

```
npm install @react-oauth/google

// main.jsx or App.jsx — wrap your app with the provider
import { GoogleOAuthProvider } from '@react-oauth/google';

<GoogleOAuthProvider clientId='YOUR_GOOGLE_CLIENT_ID'>
  <App />
</GoogleOAuthProvider>

// LoginPage.jsx
import { GoogleLogin } from '@react-oauth/google';

function LoginPage() {
  const handleSuccess = async (credentialResponse) => {
    const idToken = credentialResponse.credential; // <- this is what we need
    await exchangeForGameToken(idToken);
  };
```

```
  return <GoogleLogin onSuccess={handleSuccess} onError={() => console.log('Login
Failed')} />;
}
```

Step 2 — Exchange for a Game Token

Send the Google ID token to the Bouncer. The backend will verify it with Google, register the player in the database if this is their first login, and return your game-specific JWT.

Field	Value
Method	POST
Endpoint	/api/auth/google
Auth	None — this is the unauthenticated login endpoint

Request Body

```
{
  "idToken": "eyJhGciOiJSUzI1NiIsImtpZCI6..."
}
```

Success Response (200 OK)

```
{
  "success": true,
  "token":    "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "playerId": "123e4567-e89b-12d3-a456-426614174000",
  "username": "Alice",
  "email":    "alice@gmail.com"
}
```

Error Response (401 Unauthorized)

```
{
  "success": false,
  "error": "Authentication failed: Invalid Google token"
}
```

> *Save both token and playerId to global state (React Context, Zustand, Redux, etc.) or localStorage immediately. You will need playerId when submitting actions and token on every single request.*

Step 3 — The Golden Rule (Authorization Header)

For every HTTP request after login, include the JWT in the Authorization header. Without it, the Bouncer returns 401 and rejects the request.

```
// Create an axios instance with the header pre-configured
import axios from 'axios';

const api = axios.create({
  baseURL: 'http://localhost:8080',
});

// Add token to every request automatically
api.interceptors.request.use((config) => {
  const token = localStorage.getItem('gameToken');
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

export default api;
```

Verify Login (Optional)

You can call GET /api/auth/me at any time to verify the token is valid and retrieve the logged-in player's info. Useful on app startup to check if a stored token is still valid.

```
// GET /api/auth/me
// Headers: Authorization: Bearer <token>

// Response:
{
  "playerId": "123e4567-...",
  "username": "Alice",
  "email":    "alice@gmail.com"
}
```

□ Section 2: Real-Time WebSocket Setup

The entire game is played over a persistent WebSocket connection using the STOMP protocol over SockJS. This connection is established once after login and maintained for the entire session. SockJS provides an automatic fallback to long-polling for environments that block WebSocket traffic.

Required Libraries

```
npm install sockjs-client @stomp/stompjs
```

Connecting to the Server

Pass your JWT in the STOMP CONNECT headers. The Bouncer's WebSocket interceptor reads this header and authenticates the session. Without it, the connection is rejected immediately with no error — the session just drops.

```
import SockJS from 'sockjs-client';
import { Client } from '@stomp/stompjs';

let stompClient = null;

export function connectWebSocket(token, onConnectedCallback) {
  stompClient = new Client({
    webSocketFactory: () => new SockJS('http://localhost:8080/ws'),
    connectHeaders: {
      Authorization: `Bearer ${token}`,   // Backend requires this!
    },
    reconnectDelay: 5000,    // Auto-reconnect after 5 seconds on drop
    heartbeatIncoming: 4000, // Ping every 4s to detect dead connections
    heartbeatOutgoing: 4000,

    onConnect: (frame) => {
      console.log('WebSocket connected:', frame);
      onConnectedCallback(stompClient);
    },

    onDisconnect: () => console.log('WebSocket disconnected'),
    onStompError: (frame) => console.error('STOMP error:', frame.headers.message),
  });

  stompClient.activate();
  return stompClient;
}

export function disconnectWebSocket() {
  if (stompClient) stompClient.deactivate();
}
```

Immediate Subscriptions on Connect

As soon as the onConnect callback fires, subscribe to these two channels. Do not wait for the player to enter the lobby or game.

STOMP Channel	Purpose

STOMP Channel	Purpose
`/topic/lobby`	Public lobby broadcast — room status updates, player joins/leaves.
`/user/queue/errors`	Private error inbox — invalid action rejections sent only to the offending player.

```
// Subscribe immediately inside onConnect callback
stompClient.subscribe('/topic/lobby', (message) => {
  const event = JSON.parse(message.body);
  handleLobbyEvent(event);  // see Section 3
});

stompClient.subscribe('/user/queue/errors', (message) => {
  const error = JSON.parse(message.body);
  showToastNotification(error.reason); // show to player
});

// ADD THIS NEW BLOCK: The private Direct Mail inbox for game starts
stompClient.subscribe('/user/queue/match-start', (message) => {
    const event = JSON.parse(message.body);
    const { matchId, players } = event;

    setMatchId(matchId);
    setPlayers(players);

    // CRITICAL: Subscribe to the match battle feed NOW
    stompClient.subscribe(`/topic/match/${matchId}`, handleBattleTick);

    navigateTo('/battle');  // Switch to Battle Arena screen
});
```

ℹ *The /user/queue/... prefix is a STOMP convention for per-user private messages. The Bouncer maps this to the authenticated player's session automatically — you do not need to include the player ID in the path.*

Section 3: The Waiting Room (Lobby)

Before a game can start, four players must be grouped into a room. The lobby supports two modes: creating a private room (share a 6-letter code with friends) or quick-joining any open public room. All lobby endpoints are REST calls, not WebSocket.

A — Creating a Private Room

Field	Value
Method	POST
Endpoint	/api/lobby/create
Auth	Required — Authorization: Bearer <token>
Body	None required

The response contains a 6-character roomCode (e.g. X7K9PQ). Share this code with your 3 friends so they can join.

```
// Response
{
  "roomCode": "X7K9PQ",
  "status":   "WAITING",
  "players":  [
    { "playerId": "...", "username": "Alice", "team": 1, "turnOrder": 1 }
  ]
}
```

B — Joining a Specific Room

Field	Value
Method	POST
Endpoint	/api/lobby/join/{roomCode}
Auth	Required
Body	JSON with characterClass

The player must choose a character class before joining. The characterClass field must be EXACTLY one of the four enum values (case-sensitive, all caps).

```
// Request body
{
  "playerId":       "123e4567-...",
  "username":       "Alice",
  "characterClass": "WIZARD"   // Must be: BARBARIAN | KNIGHT | RANGER | WIZARD
}
```

C — Quick Join (Auto-Match)

Field	Value
`Method`	POST
`Endpoint`	/api/lobby/rooms/quick-join
`Auth`	Required
`Body`	Same as join — playerId, username, characterClass

Finds any open room and joins it automatically. If no rooms are available, creates a new one and returns it. The player then waits for others to join via the /topic/lobby subscription.

D — Leaving a Room

Field	Value
`Method`	DELETE
`Endpoint`	/api/lobby/rooms/{roomCode}/players/{playerId}
`Auth`	Required

E — Lobby WebSocket Events (Incoming)

While waiting in a room, your /topic/lobby subscription will receive these event types. Parse the type field first, then handle accordingly.

event.type	What it means & what you should do
`ROOM_UPDATE`	A player joined or left the room. Re-render the player list.

```
function handleLobbyEvent(event) {
  switch (event.type) {
    case 'ROOM_UPDATE':
      updatePlayerList(event.players);
      break;
  }
}}
```

> ⚠⚠ *The MATCH_START subscription to /topic/match/{matchId} must happen immediately when you receive this event. If you delay (e.g. after a screen transition animation), you may miss the first TURN_RESULT broadcast.*

□□ Section 4: The Battle Loop

The game is a 2v2 turn-based fight. Team 1 (Players 1 & 2) vs Team 2 (Players 3 & 4). Players act in turn order: P1 → P2 → P3 → P4 → P1 → ... Dead players and bound players are automatically skipped by the engine.

Submitting an Action

When it is the current player's turn, they click an attack/ability button. You send a STOMP message — not an HTTP request. The WebSocket path is faster and does not require re-establishing a connection for each action.

Field	Value
`STOMP Destination`	/app/action
`Alternative`	POST /api/game/action (HTTP fallback, same logic)
`Auth (STOMP)`	Automatically uses the token from the CONNECT header
`Auth (HTTP)`	Authorization: Bearer <token> header required

Action Payload

```
stompClient.publish({
  destination: '/app/action',
  body: JSON.stringify({
    matchId:        '123e4567-e89b-12d3-a456-426614174000',  // UUID
    actorPlayerId:  '987e6543-...',  // Must match the authenticated player
    actionType:     'RAGE_STRIKE',   // See Section 5 for full enum list
    targetPlayerId: '555e1234-...',  // null or omit for AoE and self-buffs
  })
});
```

> □ *Security: The Bouncer verifies that actorPlayerId matches the JWT token. A player cannot submit actions on behalf of another player. If this check fails, the action is silently rejected.*

Receiving the Turn Result

The Referee engine computes the result and the Bouncer broadcasts a TURN_RESULT event to all 4 players on /topic/match/{matchId} simultaneously. All clients update their UI from the same payload.

Full TURN_RESULT Payload

```
{
  "type": "TURN_RESULT",
  "matchId": "123e...",
  "data": {
    "valid":       true,
    "turnNumber":  5,
    "actionType":  "RAGE_STRIKE",
    "actorPlayerId":  "987e...",
    "targetPlayerId": "555e...",
    "damageDealt":    52,
    "healingDone":    0,
    "manaUsed":       25,
    "manaDrained":    0,
    "targetDied":     false,
    "winnerTeam":     null,
    "effectsApplied": [
      { "effectType": "BIND", "targetPlayerId": "555e...", "magnitude": 1,
"turns": 1 }
    ],
    "stateAfter": {
      "status":       "IN_PROGRESS",
      "turnNumber":   6,
      "nextTurnOrder": 2,
      "winnerTeam":   null,
      "players": [
        {
          "playerId":     "987e...",
          "username":     "Alice",
          "team":         1,
          "turnOrder":    1,
          "hp":           150,
          "maxHp":        150,
          "mana":         45,
          "maxMana":      60,
          "alive":        true,
          "activeEffects": ["ATTACK_BUFF"]
        },
        { ... },  // player 2
        { ... },  // player 3
        { ... }   // player 4
      ]
    }
  }
}
```

What To Do With The Result

Always use the stateAfter object as the source of truth for rendering. Never derive state from cumulative deltas — always replace your entire game state with stateAfter.players on every tick.

```
function handleBattleTick(message) {
```

```
  const event = JSON.parse(message.body);
  if (event.type !== 'TURN_RESULT') return;

  const { data } = event;

  // 1. Check if action was valid
  if (!data.valid) {
    showToast(`Invalid action: ${data.rejectionReason}`);
    return;
  }

  // 2. Play damage/heal animation
  if (data.damageDealt > 0) {
    showFloatingDamage(data.targetPlayerId, data.damageDealt);
  }
  if (data.healingDone > 0) {
    showFloatingHeal(data.targetPlayerId, data.healingDone);
  }

  // 3. Update ALL player health bars / mana bars from stateAfter
  //    (This is the source of truth — replace, don't patch)
  setGameState(data.stateAfter);

  // 4. Highlight whose turn it is next
  setCurrentTurnOrder(data.stateAfter.nextTurnOrder);

  // 5. Show effect badges (BIND icon, INVISIBLE shimmer, etc.)
  //    data.stateAfter.players[x].activeEffects = ['BIND', 'ATTACK_BUFF', ...]

  // 6. Show death animation if someone died
  if (data.targetDied) {
    playDeathAnimation(data.targetPlayerId);
  }

  // 7. Check for game over
  if (data.stateAfter.winnerTeam !== null) {
    const myTeam = getMyTeam(); // from your game state
    if (myTeam === data.stateAfter.winnerTeam) {
      showVictoryScreen();
    } else {
      showDefeatScreen();
    }
  }
}
```

Handling Invalid Actions

If a player submits an invalid action (wrong turn, not enough mana, targeting dead player, etc.), only that player receives an error via /user/queue/errors. All other players' UIs remain unchanged.

```
// Received on /user/queue/errors
{
  "type":   "ACTION_REJECTED",
  "reason": "Not enough mana (20/25)"
}

// OR
{
  "type":   "ACTION_REJECTED",
  "reason": "Not your turn"
}

// Common rejection reasons:
// "Not your turn"
// "Not enough mana (X/Y)"
// "Target is already dead"
// "You are bound and cannot act this turn"
// "Cannot target own teammate with damage"
// "Your class cannot use HEAL"
```

□ Section 5: Game Engine Dictionary

This section documents every enum value, effect, and rule that the backend engine enforces. Use these exact strings — they are case-sensitive.

Universal Rules

Rule	Details
`Mana Regeneration`	+10 mana every turn automatically, up to the player's maxMana.
`Effect Stacking`	Effects of the same type do NOT stack — applying a second BIND refreshes the duration.
`Invisible Miss`	If a player has INVISIBLE, the next attack targeting them deals 0 damage and removes the effect.
`Precise Shot`	PRECISE_SHOT is the only ability that ignores INVISIBLE — it also removes the effect.
`Bind Skip`	A BINDed player is skipped in turn order automatically. Bind ticks before their turn.
`Win Condition`	When all alive players on one team reach HP = 0, winnerTeam is set and the match ends.
`Player ordering`	P1 and P2 are always Team 1. P3 and P4 are always Team 2. This is assigned in the lobby.

Status Effects

When parsing stateAfter.players[x].activeEffects, you receive an array of these exact strings. Render appropriate badges or visual effects for each.

Action Type (Enum)	Mana Cost	Target	Description
BIND	—	Passive	Player's next turn is skipped entirely. 1 turn duration unless applied by BINDING_ARROW (2 turns).
INVISIBLE	—	Passive	The next incoming attack automatically misses. The effect is consumed on hit or on PRECISE_SHOT.
ATTACK_BUFF	—	Passive	Outgoing damage is increased by the magnitude percentage. Applied by BATTLE_CRY.
DEFENSE_BUFF	—	Passive	Incoming damage is reduced by the magnitude percentage. Applied by GUARDIAN_AURA.
ATTACK_DEBUFF	—	Passive	Outgoing damage is reduced by the magnitude percentage. Applied by WEAKEN.

Character Classes

Each class has a distinct role, base stats, and exclusive abilities. Present a class selection UI before a player joins a room.

□ Barbarian — The Bruiser

Base HP: 150 HP | Base Mana: 60 Mana

Action Type (Enum)	Mana Cost	Target	Description
BASIC_ATTACK	0	Single Enemy	Standard melee hit. Available to all classes.
RAGE_STRIKE	**25**	Single Enemy	Heavy blow dealing 1.5x the Barbarian's base damage.
WHIRLWIND	**40**	All Enemies	Spins and hits both enemies for 60% damage each. Send targetPlayerId as null.
BATTLE_CRY	**20**	Self	Applies ATTACK_BUFF to self for 2 turns. Damage output increases significantly. Send targetPlayerId as null.

□□ Knight — The Support Tank

Base HP: 120 HP | Base Mana: 80 Mana

Action Type (Enum)	Mana Cost	Target	Description
`BASIC_ATTACK`	0	Single Enemy	Standard melee hit.
`SHIELD_BASH`	**20**	Single Enemy	Deals 80% damage AND applies BIND for 1 turn. The target skips their next turn.
`HEAL`	**30**	Self or Teammate	Restores up to 30 HP. Cannot exceed maxHp. Can target any alive player on your team.
`GUARDIAN_AURA`	**35**	Entire Team	Applies DEFENSE_BUFF to self and teammate simultaneously. Send targetPlayerId as null.

□ Ranger — The Controller

Base HP: 90 HP | Base Mana: 110 Mana

Action Type (Enum)	Mana Cost	Target	Description
`BASIC_ATTACK`	0	Single Enemy	Standard ranged hit.
`PRECISE_SHOT`	**15**	Single Enemy	Standard damage but pierces and removes INVISIBLE. The only ability that does this.
`BINDING_ARROW`	**25**	Single Enemy	Low damage (50%) but applies BIND for 2 turns — the longest bind in the game.
`VANISH`	**20**	Self	Applies INVISIBLE to self. The next attack targeting you misses. Send targetPlayerId as null.

□ Wizard — The AoE Debuffer

Base HP: 80 HP | Base Mana: 140 Mana

Action Type (Enum)	Mana Cost	Target	Description
`BASIC_ATTACK`	0	Single Enemy	Basic magic hit.

Action Type (Enum)	Mana Cost	Target	Description
ARCANE_BOLT	30	Single Enemy	High single-target magic damage at 1.0x Wizard base.
CHAIN_LIGHTNING	45	All Enemies	Arcs between both enemies for 50% damage each. Send targetPlayerId as null.
MANA_DRAIN	20	Single Enemy	Steals up to 30 mana from the target. Deals (stolen mana / 2) as HP damage.
WEAKEN	25	Single Enemy	Applies ATTACK_DEBUFF to the target for 2 turns. Their outgoing damage is reduced.

Section 6: Post-Game & Leaderboard

After a game ends, the Scribe service processes XP and updates player stats asynchronously. You do not need to trigger this — the backend handles it automatically after winnerTeam is set. You only need to display the results.

XP Formula

Players earn XP even when they lose, scaled by their individual performance. Display earned XP prominently on the post-game screen.

XP Source	Amount
Base Win	+100 XP for being on the winning team
Base Loss	+25 XP for being on the losing team
Kill Bonus	+15 XP per enemy kill
Speed Bonus (Win)	+5 XP per turn under 20, if the match lasted fewer than 20 total turns

Fetching the Leaderboard

Field	Value
Method	GET
Endpoint	/api/scribe/leaderboard?page=0&size=20
Auth	Required

```
// Response — array of ranked player entries
[
```

Field	Value

```
  {
    "rank":     1,
    "playerId": "123e...",
    "username": "Alice",
    "wins":     10,
    "losses":   2,
    "xp":       1234,
    "winRate":  83.33
  },
  { ... }
]
```

Section 7: Quick Reference

All REST Endpoints

Action Type (Enum)	Mana Cost	Target	Description
`POST /api/auth/google`	**None**	None	Trade Google idToken for game JWT
`GET /api/auth/me`	**Bearer JWT**	None	Get logged-in player info
`POST /api/lobby/create`	**Bearer JWT**	None	Create a new private room
`POST /api/lobby/join/{roomCode}`	**Bearer JWT**	JSON body	Join a specific room
`POST /api/lobby/rooms/quick-join`	**Bearer JWT**	JSON body	Auto-match into any open room
`GET /api/lobby/rooms/{code}`	**Bearer JWT**	None	Get current room state
`DELETE /api/lobby/rooms/{code}/players/{id}`	**Bearer JWT**	None	Leave a room
`POST /api/game/action`	**Bearer JWT**	JSON body	Submit action (HTTP fallback)
`GET /api/scribe/leaderboard`	**Bearer JWT**	None	Fetch leaderboard (paginated)

All STOMP Channels

Channel	Direction	Purpose
`/app/action`	Client → Server	
`/topic/lobby`	Server → Client	
`/topic/match/{matchId}`	Server → Client	
`/user/queue/errors`	Server → Client	

All actionType Enum Values

actionType (exact string)	Class	Cost	Targeting
`BASIC_ATTACK`	All classes	0 mana	Single target
`RAGE_STRIKE`	Barbarian	25 mana	Single target
`WHIRLWIND`	Barbarian	40 mana	AoE (null target)
`BATTLE_CRY`	Barbarian	20 mana	Self (null target)
`SHIELD_BASH`	Knight	20 mana	Single target
`HEAL`	Knight	30 mana	Self or teammate
`GUARDIAN_AURA`	Knight	35 mana	AoE team buff (null target)
`PRECISE_SHOT`	Ranger	15 mana	Single target (ignores INVISIBLE)
`BINDING_ARROW`	Ranger	25 mana	Single target
`VANISH`	Ranger	20 mana	Self (null target)
`ARCANE_BOLT`	Wizard	30 mana	Single target
`CHAIN_LIGHTNING`	Wizard	45 mana	AoE (null target)
`MANA_DRAIN`	Wizard	20 mana	Single target
`WEAKEN`	Wizard	25 mana	Single target

> *Tip for UI targeting: Disable the target selector for actions where targeting is null (AoE/self-buffs). A quick check: if the actionType is WHIRLWIND, CHAIN_LIGHTNING, BATTLE_CRY, VANISH, or GUARDIAN_AURA — set targetPlayerId to null automatically.*